



AI-Based Virtual Camera Tracking for FSOC Coarse Alignment

A deterministic software laboratory for pointing, acquisition and tracking of mobile free-space optical terminals

PROBLEM	26169
ORGANIZATION	Indian Space Research Organisation (ISRO)
DEPARTMENT	Department of Space / Space Applications Centre
CATEGORY / THEME	Software / Smart Automation
SOFTWARE BASELINE	Archis 2.0.0 source commit 0143499
REPORT STATUS	Technical revision 25 September 2026

Abstract

Archis is an AI-assisted virtual camera tracking laboratory for the coarse-alignment stage of mobile free-space optical communication. It combines deterministic scene generation, explicit two-terminal 3D geometry, bounded pan-tilt actuation, optional inertial fast-mirror stabilization, full-frame acquisition, hybrid classical and ONNX-assisted localization, temporal beacon identity verification, six-state Kalman estimation, and truth-separated evaluation. The system also analyzes native-resolution MP4 and image sequences with optional truth sidecars. This revision uses freshly generated, verified 27,000-frame evidence from the current local source tree.

116 / 116

TESTS PASSING

27,000

MATRIX FRAMES

15 / 15

ACCURACY-CONTROL PASS

15 / 15

HOST FPS PASS

Engine fingerprint: ad8cd198e6eebf589b795f8c5b69f79dc60663bd270a16797f9c4582a0d227a

SCOPE, OUTCOMES AND EVIDENCE

1 Executive Summary

The coarse-alignment problem is to keep a remote optical beacon inside a narrow camera field of view despite terminal motion, platform vibration, atmospheric degradation, sensor noise and temporary loss. Archis supplies a repeatable software environment for developing and evaluating that loop without specialist optical hardware. The application separates simulation truth from perception inputs, preserving an honest measurement boundary while still giving evaluators complete ground-truth metrics.

Current verified result. The production Hybrid profile acquired 15 of 15 matrix runs inside the 2.0 s gate. Centroid RMSE reached at most 2.59 px. 15 runs passed every configured accuracy/control gate, and conservative throughput met the host gate in 15 runs. Platform-jitter trials use a declared virtual inertial sensor and bounded fast mirror; they are not unassisted pedestal-only claims.

What is implemented

SIMULATION	TRACKING	EVIDENCE
- Seeded 30 Hz world with fixed simulation clock - Moving designated target plus decoy terminals - Explicit RX/TX 3D pose, range, velocity and orientation - Noise, jitter, atmosphere, turbulence and dropout	- Full-frame robust acquisition before Kalman gating - Hybrid NanoSpot plus classical evidence - SEARCH to REACQUIRE canonical state machine - PID, anti-windup, limits and feed-forward	- Truth-based centroid and pointing metrics - p50, p95 and maximum processing latency - CSV, JSON and HTML session reports - A/B, stress, optical and velocity validation

Configured release gates

Metric	Gate	Meaning
Acquisition time	≤ 2.0 s	First confirmed TRACK state
Tracking error	≤ 10 px	Centroid and camera pointing evaluated separately
Target loss	$< 5\%$	Fraction of truth-visible frames without lock
Reacquisition	≤ 1.0 s	Worst recovery after recoverable loss
Processing	≥ 20 FPS	Conservative 1000 / p95 latency on the measured host

These are project release gates encoded in the current configuration; they are not substituted for any unavailable official parameter table.

Report map

SECTION	TOPIC	PAGE
2	Problem formulation and two-terminal world	3
3	System architecture and data contracts	4
4	Deterministic simulation and disturbances	5
5	Perception and target acquisition	6
6	Optical evidence and adaptive estimation	7
7	Identity, association and state logic	8
8	Control, camera and external media	9
9	Verification methodology and test coverage	10
10	Benchmark results	11
11	Optical validation	12
12	Velocity envelope and limitations	13
13	Delivery readiness and conclusions	14

2 Problem Formulation and Two-Terminal World

Free-space optical communication uses a narrow optical beam, so link establishment depends on pointing, acquisition and tracking (PAT). Coarse alignment must first discover the designated terminal, identify it among distractors, estimate its image-plane motion, and steer the receiving camera until the target remains near the optical axis. Fine pointing and the communication link then operate inside that capture envelope.

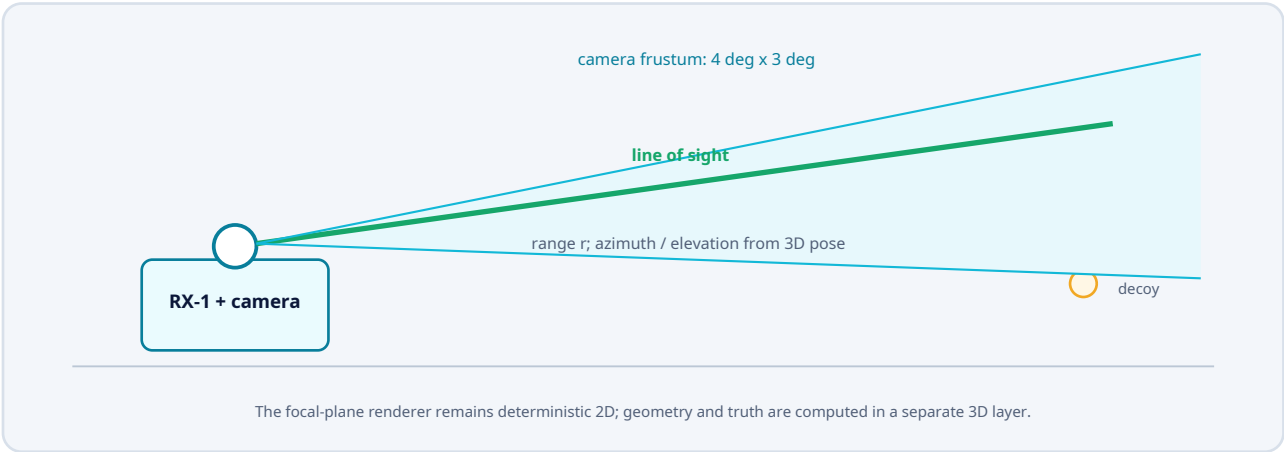


Figure 1. Explicit receiver, transmitter, decoy and camera-frustum representation.

3D geometry

A right-handed terminal state stores position $p = (x, y, z)$, velocity v , and yaw-pitch-roll orientation. For relative line-of-sight vector $d = p_{TV} - p_{DV}$, the implementation computes:

range $r = \sqrt{dx^2 + dy^2 + dz^2}$ azimuth = $\text{atan2}(dx, dz)$ elevation = $\text{atan2}(dy, \sqrt{dx^2 + dz^2})$

Relative azimuth and elevation subtract camera boresight. The transmitter is in view when both offsets lie within half of the horizontal and vertical FOV. World-plane target motion is mapped to a physical range plane, so telemetry includes separation, line-of-sight angles, physical velocity, terminal orientation, decoys and in-FOV status.

Camera projection

For viewport width W , height H and fields of view θ_x, θ_y , the pinhole intrinsics are $f_x = (W/2)/\tan(\theta_x/2)$ and $f_y = (H/2)/\tan(\theta_y/2)$. Sensor offsets are converted to angular error before the controller acts. The default camera is 640 x 480 pixels, 4 x 3 degrees, at 30 Hz.

Model boundary. This is an explicit 3D two-terminal geometry and telemetry model, not a full ray-traced optical propagation engine. The actual camera frame remains a deterministic 2D focal-plane renderer. A visible line in a world view is therefore a visualization of line of sight or beam intent, not a claim that a camera sees the propagating laser from the side.

Functional objective

Required capability	Archis implementation
Configurable environment and moving targets	Versioned scenarios, presets, trajectories and decoys
Movable camera	Rate- and acceleration-limited virtual pan-tilt camera
Automatic detection and continuous tracking	Hybrid acquisition, Kalman estimation and explicit state machine
Atmosphere, vibration, motion and noise	Ordered, seeded disturbance pipeline with dropout and turbulence
Real-time statistics and performance log	Dashboard plus CSV, JSON and HTML evidence

MODULAR, DETERMINISTIC AND UI-COMPATIBLE

3 System Architecture and Data Contracts

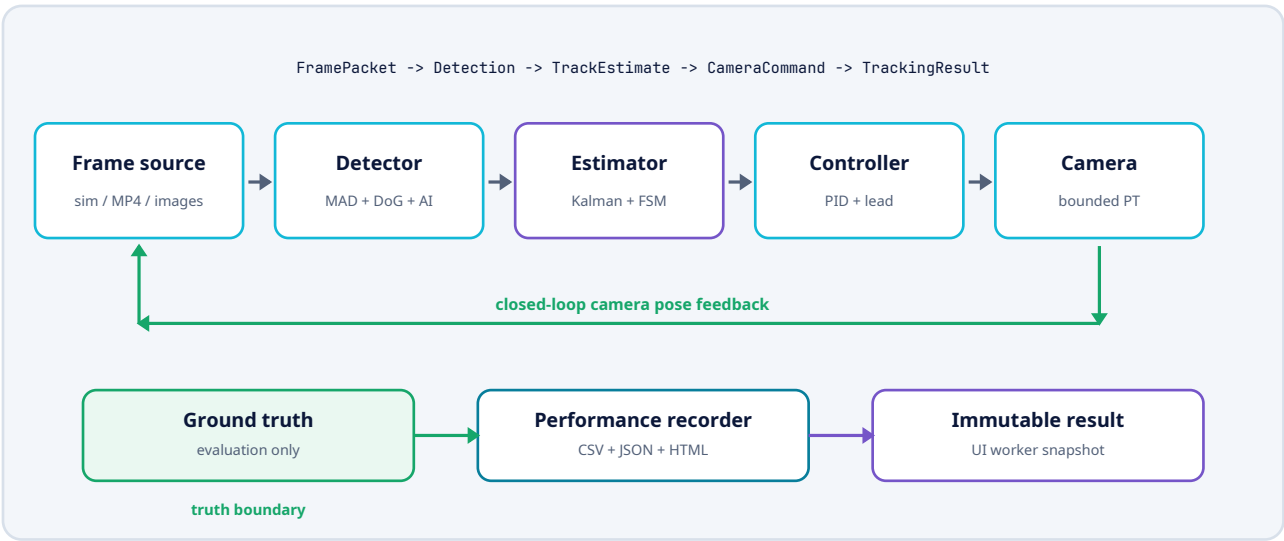


Figure 2. Tracking dataflow and the one-way ground-truth evaluation boundary.

The public TrackingSystem facade preserves the established PyQt/Fluent workflow while the internal engine uses immutable snapshots. This lets the worker thread process frames without mutating data being painted by the UI, and it lets the CLI run the same engine headlessly.

Contract	Essential contents	Role
FramePacket	Frame index, timestamp, image, source metadata	Common input from simulator, MP4, image or sequence
GroundTruthSample	Target centroid, visibility, identity	Evaluation only; never consumed by perception or control
Detection	Centroid, size, confidence, algorithm and optical evidence	Candidate observation
TrackEstimate	Position, velocity, covariance, confidence and state	Filtered kinematic state
CameraCommand	Pan and tilt rate commands	Bounded actuator request
TrackingResult	Frame, candidates, selection, estimate, command, timings	Immutable output for UI and reporting

Software modules

Subsystem	Modules	Primary responsibility
World and input	world_model, environment, target, sources	Generate scenes or ingest native media
Perception	detector, ai_detector, photometry, codelock	Propose, refine and identify beacons
Autonomy	kalman_filter, state_machine, controller, camera, gimbal	Estimate motion; command bounded pedestal and optional fast mirror
Evaluation	performance, telemetry, demo_evidence	Compute metrics and export replayable evidence
Interfaces	ui package, cli, scenario, presets	Operate and validate the same tracking engine

Frame sources

A common source abstraction supports deterministic simulation, native-resolution MP4, a single image and ordered image sequences. External inputs bypass simulated camera-pose feedback because the video cannot be physically re-pointed; Archis still emits theoretical camera commands and operational tracking metrics. With no truth sidecar, reports are labelled *observed tracking* and do not claim centroid accuracy.

Scenario schema and compatibility

Versioned JSON scenarios define camera, world, terminal targets, disturbances, controller, detector, duration and random seed. Legacy presets migrate through a validated adapter and retain the current names: Nominal LEO Pass, Evasive Target, Heavy Turbulence, Cloud Dropout and Platform Jitter. Invalid edits are rejected atomically instead of leaving a partially updated engine.

Truth isolation

The simulator computes sensor-space truth after camera geometry and disturbances. Only the recorder receives it. Detector, association, Kalman filter and controller operate on pixels and prior state. Tests replace or remove truth and verify that tracking decisions do not change.

The desktop UI shows the active scenario, World/Sensor views and live metrics. The CLI runs the same core for simulation, video analysis, benchmarking, comparison, stress sweeps and optical validation.

REPEATABLE PIXELS, REALISTIC STRESSORS

4 Deterministic Simulation and Disturbances

A fixed simulation clock advances the world independently of wall-clock rendering. Every stochastic subsystem receives a stable per-session generator derived from the scenario seed; burst loss has its own stream so enabling it does not silently change unrelated sensor noise. Two runs with identical scenario, seed and engine fingerprint therefore reproduce the same accuracy and state metrics.

Ordered image formation

ORDER	STAGE	IMPLEMENTATION
1	World and terminals	Render background, primary beacon and decoy terminals
2	Camera pose	Crop/project the world through current pan-tilt boresight
3	Platform motion	Apply deterministic platform displacement and jitter
4	Atmosphere	Contrast, blur, warp and irradiance scintillation
5	Sensor	Gaussian, Poisson and salt-and-pepper noise; clipping
6	Channel loss	Scheduled blackout or seeded burst dropout

Disturbance models

Disturbance	Configuration	Purpose
Camera jitter	Maximum pixel displacement	High-frequency line-of-sight motion
Platform motion	Linear, circular, random, spiral or figure-eight	Lower-frequency moving-platform dynamics
Atmospheric visibility	Clear, haze, fog, rain or low light plus severity	Bulk contrast and brightness loss
Turbulence	Warp px, blur sigma, scintillation model	Angle-of-arrival, seeing and irradiance variation
Illumination	Flicker fraction and frequency	Source/background intensity variation
Sensor noise	Gaussian sigma, Poisson and salt-pepper ratio	Detector stress
Dropout	Timed window or mean clear/loss dwell times	Cloud blockage and burst losses

Turbulence and scintillation

The compatibility default uses seeded log-normal scintillation. An optional Gamma-Gamma mode derives shape parameters from Rytov variance and samples irradiance as the product of two normalized gamma variates. This adds a physically parameterized strong-fluctuation stress mode without altering existing presets. Spatial warp and Gaussian blur remain separately configurable so angle-of-arrival, seeing and irradiance effects can be isolated.

Gamma-Gamma: $I = X Y$, with $X \sim \text{Gamma}(\alpha, 1/\alpha)$, $Y \sim \text{Gamma}(\beta, 1/\beta)$, so $E[I] = 1$. Alpha and beta are bounded functions of the configured Rytov variance.

Camera dynamics

The virtual gimbal applies commanded angular rates subject to independent pan/tilt limits and acceleration bounds. Both position and the internal dynamic state are clamped at camera boundaries. For the platform-jitter preset only, an optional seeded inertial attitude measurement (0.5 px standard deviation) drives the existing fast-mirror actuator within a +/-0.5 deg throw to counter measured platform motion before each virtual exposure. This is an explicitly assisted sensor model, not ground-truth beacon feedback and not a claim that the slow pedestal alone rejects high-frequency vibration. The inertial noise generator is separate from image noise, preserving reproducibility. Gyro telemetry, boresight, line-of-sight and commanded rates remain distinct.

Determinism applies to generated pixels, truth, state transitions and accuracy metrics. Processing time is wall-clock data and remains host-dependent by design.

A HYBRID CASCADE WITH HONEST FALLBACKS

5 Perception and Target Acquisition

Initial acquisition cannot assume a valid Kalman prediction. Archis therefore scans the full frame during SEARCH, ACQUIRE and global REACQUIRE. Prediction gating becomes active only after a confirmed track, with periodic full-frame safety scans retained to detect divergence and support decoy handling.

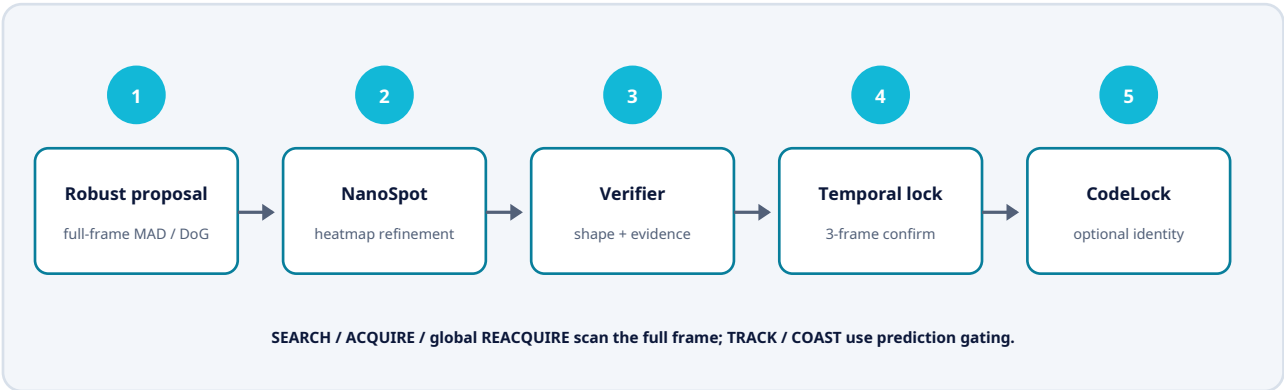


Figure 3. Default Hybrid cascade; CodeLock is active only when a beacon pattern is configured.

Robust candidate proposal

The classical front end estimates background with robust statistics, uses the median absolute deviation (MAD) for noise scale, and applies Difference-of-Gaussians enhancement. Connected components produce multiple candidates instead of collapsing the image to one threshold peak. Candidate area, morphology, centroid padding and prediction gates scale with measured spot FWHM rather than fixed pixel constants.

Subpixel localization

For non-negative background-subtracted intensities I_i at pixel coordinates (x_i, y_i) , intensity-weighted centroiding computes $\hat{x} = \text{sum}(I_i x_i) / \text{sum}(I_i)$ and $\hat{y}$ equivalently. The hybrid path also refines the NanoSpot heatmap maximum over a local 7×7 window. Each selected candidate carries algorithm identity, AI score, optical evidence and optional CodeLock correlation.

NanoSpot model

The bundled ONNX model consumes a 64×64 normalized grayscale patch and produces a heatmap used for localization and candidate verification through OpenCV DNN. Model provenance and calibration metadata ship with the executable. Missing, corrupt or failed inference is reported explicitly; the application never relabels a classical fallback as AI.

AI training and threshold calibration

Bundled metadata identifies a synthetic-trained CNN: 21,600 training examples over 900 steps and 240 validation examples. A separate deterministic 400-sample threshold exercise splits synthetic positives and negatives 70/30 for calibration and held-out checking. The shipped AI-only threshold is 0.88; held-out false locks were 0 in this small synthetic set, but recall was about 30%. These data do not establish performance on unseen judge videos or real optical hardware.

Mode	Purpose	Status
Hybrid	Robust proposals + NanoSpot evidence + temporal confirmation + optional CodeLock	Production default
IWC	Intensity-weighted centroid	Classical A/B baseline
Gaussian Fit	Two-dimensional Gaussian surface fit	Demonstration / comparison
NCC	Normalized cross-correlation	Template baseline
AI ONNX	NanoSpot-only detection path	Experimental; conservative recall

Important limitation. AI-only is not presented as the best operating mode. Earlier comparison runs showed zero lock for the conservative AI-only threshold on clean synthetic data, while Hybrid remained reliable. The AI path supplies evidence inside the production cascade; its threshold must be recalibrated before any standalone deployment claim.

Association

When several candidates exist, statistical association ranks them using the predicted Kalman innovation and covariance. A wide second-stage catastrophic-jump gate protects estimator state even during periodic global scans. Designated-target controls and temporal identity evidence prevent the brightest decoy from automatically becoming the track.

FROM MEASURED SPOT QUALITY TO FILTER COVARIANCE

6 Optical Evidence and Adaptive Estimation

Detector confidence alone is not a physical noise measurement. Archis therefore measures the selected optical spot and converts those measurements into bounded Kalman observation uncertainty.

Field	Computation	Use
FWHM	Equivalent circular diameter of the seed's half-maximum component	Spot scale and candidate geometry
Aperture SNR	Background-subtracted aperture signal divided by robust annulus noise	Measurement-noise model
Peak SNR	Peak above local background divided by robust noise sigma	Diagnostic evidence
Clipping	Candidate bounding box touches a sensor boundary	Adds 2 px bias in quadrature
Saturation	Fraction of half-height core at digital ceiling	Adds 0.2 px bias above 5%
Background	Annulus median and 1.4826 x MAD with quantization floor	Robust local noise estimate

Measurement covariance

The calibrated base uncertainty is $\sigma_z = k \text{ FWHM} / (2 \text{ SNR}_{\text{aperture}})$. The shipped profile uses $k = 14.2$, the rounded conservative 95th-percentile fit from the 54-case validation matrix. Sigma is clamped to 0.15-20 px; clipping and saturation penalties are added in quadrature. The position block of R uses σ_z^2 .

```
sigma = clamp(sqrt(clamp(k * FWHM / (2 * SNR), 0.15, 20)^2 + b_clip^2 + b_sat^2), 0.15, 20)
```

Six-state acceleration-aware Kalman filter

The implemented state is [x, vx, ax, y, vy, ay]. A discrete constant-acceleration transition projects position and velocity using the measured simulation step. Configurable process noise captures unmodelled maneuvers. Position measurements use adaptive R from optical evidence, or confidence-based R when photometry is unavailable. Correction uses Joseph form, $P = (I - KH)P(I - KH)^T + KRK^T$, followed by symmetry restoration. Candidate association and a wider innovation gate reject implausible jumps.

Why scale-relative geometry matters

A fixed blob area and fixed association radius fail when range, blur or turbulence changes apparent spot size. Measured FWHM now controls DoG sigmas, area bounds, padding and the local prediction gate within configured minimum and maximum scale limits. The result adapts to 3-15 px validation spots without changing the underlying tracker mode.

Evidence-to-estimate flow

PIXEL EVIDENCE	UNCERTAINTY	FILTER EFFECT
Narrow, high-SNR, unclipped spot	Small bounded sigma	More weight on current observation
Broad or noisy spot	Larger sigma	More weight on motion prediction
Clipped / saturated spot	Bias penalty in quadrature	Avoid overconfidence near sensor limits
No valid observation	No correction	COAST prediction, then reacquisition

The optical-quality model is calibrated on deterministic synthetic spots. It is a software-validation model, not a substitute for laboratory calibration of a specific telescope, detector, gain chain and point-spread function.

ACQUISITION THAT KNOWS WHEN NOT TO LOCK

7 Identity, Association and Tracking State

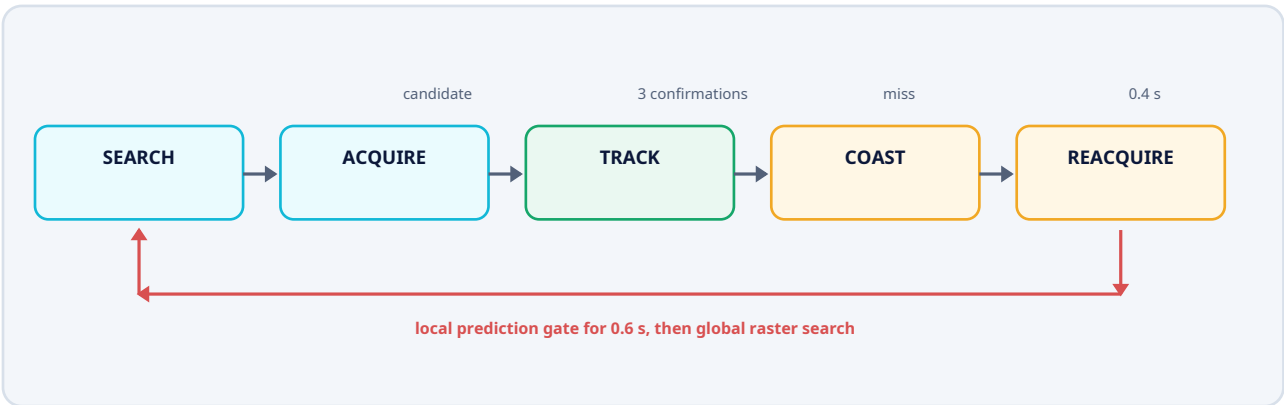


Figure 4. Canonical state machine and default timing.

State behavior

State	Search scope	Transition intent
SEARCH	Full frame	Any plausible candidate starts ACQUIRE
ACQUIRE	Full frame	Three confirmed detections promote to TRACK; excess misses return to SEARCH
TRACK	Prediction gated + periodic safety scan	A miss enters COAST
COAST	Local prediction gate	Recovery returns to TRACK; 0.4 s timeout enters REACQUIRE
REACQUIRE	Local for 0.6 s, then global	A candidate restarts temporal ACQUIRE

CodeLock beacon identity

When a coded beacon is configured, Archis samples local background-subtracted intensity around every associated candidate. A per-candidate history is compared with a repeated binary pattern using phase-invariant normalized correlation. Patterns must contain both symbols and at least seven bits; the default acceptance threshold is 0.70. Candidates are not released to the tracker until enough samples exist and correlation passes.

$$\rho = \max_phase \left((s - \text{mean}(s)) \cdot (c_phase - \text{mean}(c)) \right) / (||s - \text{mean}(s)|| \cdot ||c_phase - \text{mean}(c)||)$$

CodeLock therefore addresses identity over time rather than merely preferring the brightest point. Stable track IDs use spatial association across the code window, and verified correlation is fused with detector confidence in the final detection contract.

Reacquisition and dropout

Short gaps are bridged by Kalman prediction in COAST. A longer gap triggers local reacquisition around the extrapolated track, then global full-frame search. Scheduled blackout parameters and two-state burst-loss dwell times come from the scenario and are recorded in reports; there is no hidden fixed-blackout benchmark shortcut.

Decoy defenses

Layer	Failure mode addressed
Multi-candidate proposal	Avoid discarding the true beacon because a decoy is brighter
Shape / AI evidence	Reject morphologically implausible highlights
Mahalanobis association	Prefer observations consistent with predicted motion
Catastrophic-jump gate	Prevent estimator corruption during global scans
CodeLock correlation	Reject a bright candidate with the wrong temporal identity
Temporal confirmation	Reject one-frame impulsive noise and transient artifacts

CodeLock is a simulated temporal intensity-code verifier. It is not a cryptographic authenticator and should not be described as one.

CLOSED-LOOP POINTING WITH BOUNDED ACTUATION

8 Control, Camera and External Media

Error and command generation

For estimated target centroid (x-hat, y-hat) and image center (c_x, c_y), image error is e_x = x-hat - c_x and e_y = y-hat - c_y. Pixel error is converted to angular error from camera FOV and viewport dimensions. Pan and tilt each use configurable PID control plus filtered velocity feed-forward:

```
u = Kp * e + Ki * integral(e) + Kd * de/dt + Kff * v; command = rate_limit(acceleration_limit(u))
```

The integrator has anti-windup bounds, derivative and feed-forward terms are smoothed, and maximum rate / acceleration are taken from configuration. Known pipeline delay can optionally project position, velocity and acceleration by a configured horizon before control. Latency compensation defaults to zero rather than pretending a fixed delay exists.

An optional inner stabilization loop is separate from the coarse PID: a simulated noisy inertial attitude readout estimates platform offset, and the bounded fast mirror opposes it. Only the Platform Jitter scenario enables this path in the reported matrix. MP4 mode has no simulated platform attitude signal and bypasses this feedback.

Default parameter	Value	Default parameter	Value
Pan / tilt Kp	8.0	Ki	0.18
Kd	0.04	Velocity feed-forward	0.8
Deadband	0.5 px	Anti-windup	3.0
Maximum rate	5 deg/s	Maximum acceleration	25 deg/s^2
Coast / local reacquire	0.4 / 0.6 s	Periodic full scan	30 frames

External media

MP4 and image-sequence sources preserve native resolution. CSV truth rows may use frame_index or timestamp_s with x_px, y_px and optional visible / target_id; JSON accepts the equivalent array or a {frames: [...]} document. Exact frame matching is the default. Timestamp interpolation is opt-in. Missing or invisible truth is retained in the log but excluded from centroid-error calculations.

Because a recorded video cannot respond to a simulated gimbal, external mode reports commands as theoretical. With sidecar truth it can score detection accuracy; without truth it reports lock behavior and processing performance only.

Real-time user interface

Surface	Information
Sensor viewport	Frame, candidates, selected target, prediction gate and boresight
Active configuration sidebar	Only current environment, source, target, disturbance, detector and controller settings
Inspector	Detector/model, evidence, centroid, CodeLock, state and pass/fail values
Charts	Centroid error, pointing offset, gimbal kinematics, processing speed and jitter FFT
Evaluator actions	Benchmark, algorithm comparison, stress sweep and evidence export

Legend labels are positioned below chart traces so live signals do not obscure centroid and pointing labels. Engine processing runs on a worker thread and returns immutable snapshots, preventing the 30 Hz tracker from blocking Qt rendering.

EVIDENCE DESIGNED TO BE REGENERATED

9 Verification Methodology and Test Coverage

Verification separates deterministic algorithm behavior from machine-dependent speed. Every generated report records scenario, seed, application version, model metadata and source fingerprint. Baseline and candidate algorithms receive identical saved frames and seeds for comparison.

Metric definitions

Metric	Definition
Centroid error	Euclidean distance between selected detection and sensor-space truth on valid visible frames
Pointing offset	Distance from truth centroid to the camera optical-axis center
RMSE / p95 / maximum	Distribution summaries after first confirmed acquisition on eligible truth-scored frames
Acquisition time	Time from source start to first confirmed TRACK state
Lock retention / loss	Tracked and accurate fraction after acquisition; loss is its complement
Reacquisition	Recovery intervals after a previously established lock
Conservative throughput	1000 divided by p95 processing latency in milliseconds

Automated tests

Area	Cases	Coverage
AI detector	14	Real inference, failures, provenance, heatmaps and decoys
Association / camera / controller / detector	10	Statistics, geometry, bounded stabilization, search and noise
Disturbances / external sources / gimbal	13	Seeded effects, phase-screen equivalence, inertial readout and media
Optics and optical validation	12	Intrinsics, FWHM/SNR, adaptive R, Gamma-Gamma, p95 and velocity
Performance / telemetry / reports	11	Truth metrics, observed mode, fingerprints and exports
UI / presets	19	Worker behavior, controls, sidebar, legends and stabilization presets
Targets and trajectories	10	Shapes, six trajectories, continuity and bounce
Unified core	19	Contracts, states, seeds, truth, CodeLock and estimator features
Two-terminal world	6	Pose, range, physical velocity, decoys and truth isolation
Demo evidence	2	Synchronized World/Sensor capture and detector gallery
Total	116	116 pass in the current headless suite

Current test run: 116 passed. The previously intermittent Review-page button-width failure was addressed with an explicit minimum width; the full headless suite and targeted layout test pass. This is local Linux evidence, not yet a clean Windows installer smoke test.

Evidence matrix

The current local engine was rerun on five 60-second scenarios at 30 Hz across seeds 26169, 26170 and 26171: 1,800 frames per run, 15 runs and 27,000 frames total. The regenerated repository evidence includes matrix CSV/JSON, canonical-seed per-frame reports, a noise sweep, host provenance and a content-hash manifest. The built-in freshness verifier passes against the current local source-tree fingerprint.

Recorded host

CPU	Operating system	Python	NumPy	OpenCV
Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz	Linux-7.2.5-3-omarchy-x86_64-with-glibc2.44	3.14.7	2.5.3	5.0.0

The report does not promote host FPS to an algorithmic invariant. Accuracy fingerprints exclude processing latency, while all latency and throughput values retain the measured host context.

FRESH VERIFIED MATRIX FROM CURRENT LOCAL SOURCE

10 Benchmark Results

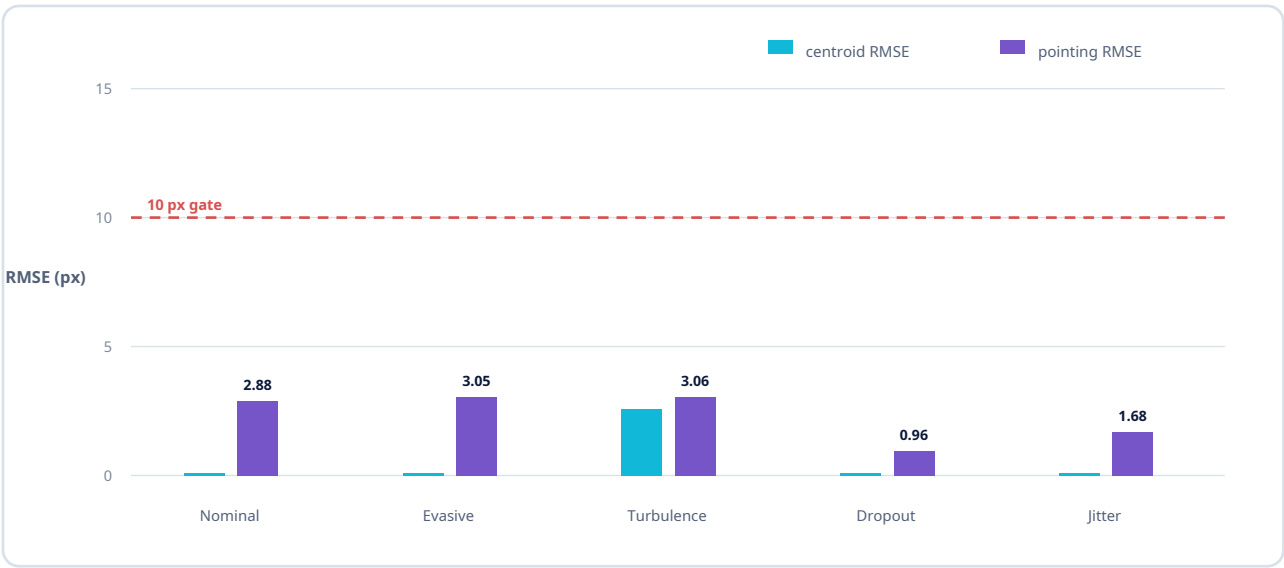


Figure 5. Mean centroid and pointing RMSE across three fixed seeds; the red line is the 10 px project gate.

Scenario	Centroid px	Pointing px	Loss %	Reacq s	A/C pass	Cons. FPS	FPS pass
Nominal LEO	0.023-0.029	2.866-2.894	0.00-0.00	n/a	3/3	96.7-187.9	3/3
Evasive target	0.029-0.031	2.972-3.184	0.00-0.00	n/a	3/3	89.9-169.2	3/3
Heavy turbulence	2.554-2.589	3.038-3.083	0.00-0.00	n/a	3/3	28.4-31.8	3/3
Cloud dropout	0.099-0.102	0.958-0.966	0.50-0.50	0.300	3/3	66.8-144.4	3/3
Platform jitter	0.025-0.037	1.681-1.687	0.00-0.00	n/a	3/3	77.2-165.8	3/3

Interpretation

- Acquisition passes in 15 of 15 runs; observed times span 0.10-0.10 s.
- Heavy-turbulence centroid RMSE is 2.55-2.59 px, below the configured 10 px gate.
- Cloud-dropout runs lose at most 0.50% of truth-visible frames and recover in at most 0.30 s.
- Platform-jitter pointing RMSE is 1.68-1.69 px with the explicitly enabled inertial sensor and bounded fast mirror; the prior unassisted run measured 13.06-13.23 px.
- Heavy-turbulence conservative throughput spans 28.4-31.8 FPS on the recorded host; host speed is not a deterministic benchmark.

Scorecard

15 / 15 ACQUISITION PASS	15 / 15 CENTROID PASS	15 / 15 POINTING PASS	15 / 15 CONSERVATIVE FPS PASS
-----------------------------	--------------------------	--------------------------	----------------------------------

Repeatability and provenance

The current local engine fingerprint is ad8cd198e6eebf589b795f8c5b69f79dc60663bd270a16797f9c4582a0d227a. Canonical-seed per-frame CSV/JSON, matrix summaries, host provenance and content hashes were regenerated from this source state. The repository manifest and seven benchmark documents now pass scripts/generate_benchmark_evidence.py --verify.

Qualification boundary. All 15 measured Linux-host runs pass the configured gates, but the jitter result depends on a simulated inertial sensor and fast mirror. Conservative FPS must still be checked on the designated Windows machine, and unseen judge MP4 performance requires an independent video with sidecar truth.

SPOT SIZE, NOISE, SATURATION AND CALIBRATION

11 Optical Validation

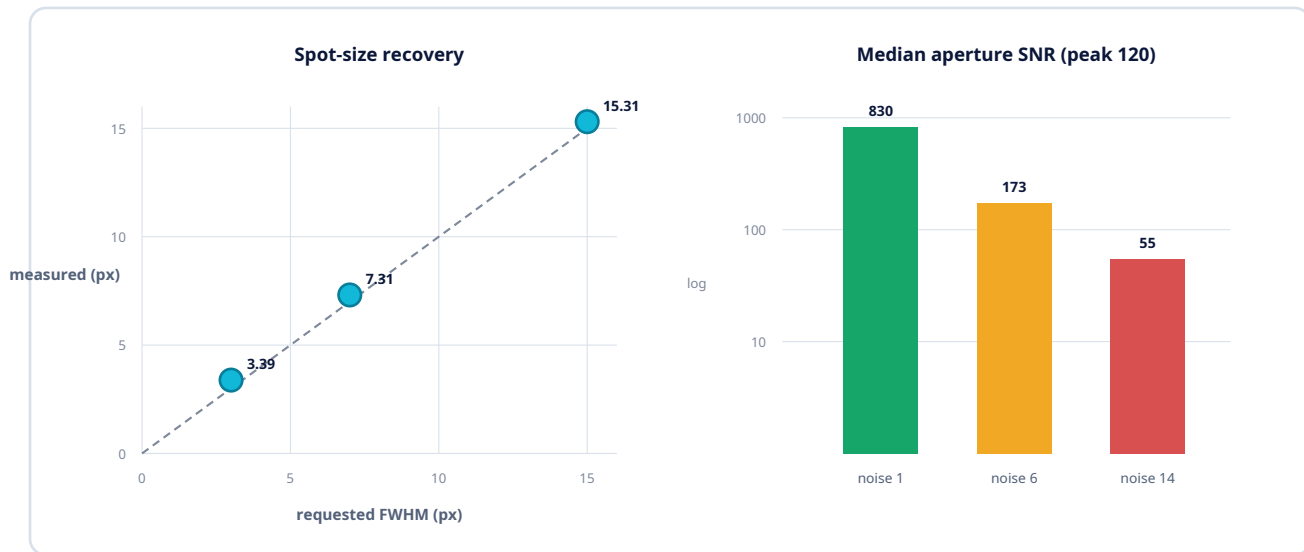


Figure 6. FWHM recovery and aperture-SNR trend from the seeded 54-case optical sweep.

The sweep covers requested FWHM 3, 7 and 15 px; Gaussian noise sigma 1, 6 and 14; requested peaks 120, 240 and 360; and two subpixel phases. All inputs are generated from seed 26169. Saturated, clipped and missed observations are excluded from fitting the measurement-noise calibration.

Requested FWHM	Median measured	Noise	Peak	Aperture SNR	Centroid error
3 px	3.39 px	1	120	379.7	0.027 px
3 px	3.19 px	6	120	64.6	0.231 px
3 px	3.65 px	14	120	27.2	0.537 px
7 px	7.31 px	1	120	1161.8	0.020 px
7 px	7.00 px	6	120	173.0	0.096 px
7 px	7.48 px	14	120	55.2	0.344 px
15 px	15.20 px	1	120	1538.0	0.017 px
15 px	15.41 px	6	120	263.3	0.176 px
15 px	14.90 px	14	120	133.6	0.246 px

Calibration result

The raw conservative fit is $k = 14.1537$; the shipped value is 14.2. It is the 95th percentile of $2 \times \text{SNR} \times \text{observed centroid error} / \text{measured FWHM}$ for eligible unsaturated observations, clipped to a defensible 1-20 range.

Saturation behavior

The 120-peak cases have median saturation fraction 0.000; the 360-peak cases have median 0.325 and maximum 0.365. The measurement model therefore records saturation explicitly and inflates uncertainty instead of treating a clipped core as perfect evidence.

The validation confirms expected trends: measured FWHM follows requested scale, aperture SNR falls as noise rises, and centroid error grows in low-SNR cases. It does not establish performance for every real optical point-spread function.

Reproduce

```
python -m archis_tracker.cli validate-optics --seed 26169 --output-dir reports/optics
```

BOUNDED CLAIMS INSTEAD OF UNBOUNDED SPEED

12 Velocity Envelope and Remaining Limitations

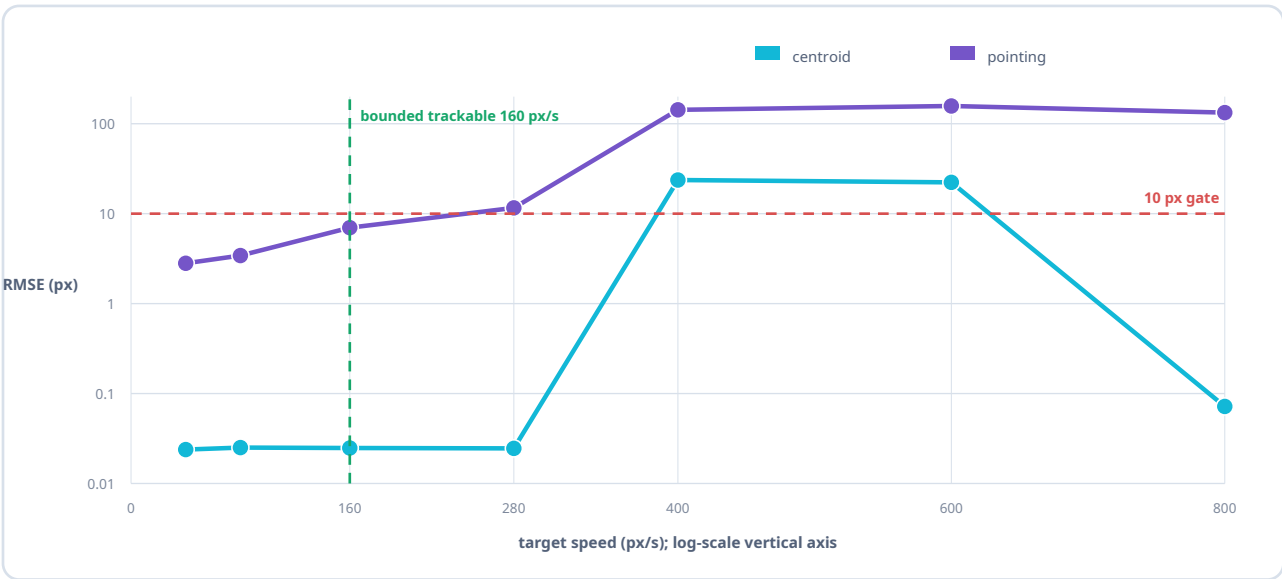


Figure 7. Nominal 300-frame velocity sweep. Vertical scale is logarithmic.

The default 5 deg/s gimbal and 160 px/deg image scale imply an 800 px/s mechanical ceiling. The measured tracking envelope is smaller because acceleration, controller dynamics, FOV and the 10 px pointing gate also matter. On the current nominal 300-frame protocol, 160 px/s is the largest passing tested speed. At 280 px/s, detection remains accurate but pointing RMSE reaches 11.59 px; higher speeds produce substantial loss.

Speed	Mech. fraction	Centroid	Pointing	Loss	Trackable
40	5%	0.024	2.81	0.00%	PASS
80	10%	0.025	3.42	0.00%	PASS
160	20%	0.025	6.99	0.00%	PASS
280	35%	0.025	11.59	0.00%	FAIL
400	50%	23.661	142.74	72.82%	FAIL
600	75%	22.287	157.46	82.89%	FAIL
800	100%	0.072	132.97	87.92%	FAIL

Future improvements and engineering priorities

Priority	Limitation	Planned response
P0	Windows 11 x64 release throughput is not measured	Run the complete matrix and clean-host installer smoke test on the reference machine
P0	Jitter passing result uses a simulated inertial aid	Validate sensor noise, latency, bandwidth and fast-mirror limits with hardware-in-the-loop
P0	Unseen judge MP4 remains unverified	Score independent noisy 30 FPS video against a withheld truth sidecar
P1	One earlier Review layout run was intermittent	Repeat UI resizing and navigation on Windows at minimum width
P1	AI-only conservative recall	Train and calibrate on held-out synthetic plus real captures; preserve Hybrid fallback
P1	3D layer is geometric, not radiometric ray tracing	Add aperture, divergence, range loss and detector response as an optional validated optical link model
P1	No laboratory hardware-in-the-loop evidence	Add camera / gimbal adapters, timestamps and repeatable replay before deployment claims
P2	Velocity bound is protocol-specific	Repeat across longer durations, initial offsets, acceleration profiles and all disturbance presets

The 160 px/s figure is an evidence-bounded result for the stated scenario and 300-frame protocol, not a universal terminal-speed rating.

Reproduce

```
python -m archis_tracker.cli velocity-envelope archis_tracker/presets/nominal_leo.json --frames 300 --output-dir reports/velocity
```

CURRENT SOFTWARE STATUS AND NEXT VALIDATION GATES

13 Submission Readiness and Conclusions

Deliverable coverage

Deliverable	Current implementation	Status
Software application	PyQt/Fluent desktop UI, headless CLI and deterministic engine	Implemented
Source code	Modular Python package, versioned scenarios, 116 collected tests	116 pass locally
Technical report	This 10-15 page revision: problem, architecture, modules, methods, testing, results and future work	Updated
User manual	Installation, operation, scenario, media, truth and report workflow	Separate deliverable
Performance log	Per-frame CSV plus JSON and branded HTML summary	Implemented
Windows package	PyInstaller one-folder workflow, portable ZIP / installer manifest path	Workflow present; clean-host smoke test remains a release gate

Evidence package contents

- Versioned scenario and random seed; application, model and engine fingerprints.
- Per-frame timestamp, state, truth eligibility, candidates, selected estimate, camera command, centroid error, pointing offset and processing latency.
- Summary RMSE / mean / p95 / maximum errors, acquisition, lock, loss, reacquisition and conservative throughput.
- Generated benchmark matrix, optical sweep, velocity envelope, provenance and verified evidence manifest.

Release procedure

Gate	Action
Correctness	Keep all 116 collected tests passing on Linux and the Windows reference host
Evidence freshness	Re-run scripts/generate_benchmark_evidence.py --verify after every engine edit
Scenario matrix	Repeat five scenarios x three seeds x 1,800 frames on Windows 11 x64
Windows delivery	Build one-folder package; smoke-test GUI, reports and resources on clean Windows 11 x64 without Python
Release integrity	Create portable ZIP and installer from the same tested tree; publish SHA-256 checksums and manifest

Conclusion

Archis now provides the complete software structure needed for FSOC coarse-alignment algorithm development: explicit terminal geometry, deterministic disturbances, robust acquisition, identity-aware tracking, adaptive estimation, bounded camera control, native external-media analysis and auditable evidence. Its strongest result is not a single low error number; it is the ability to explain why a run passed or failed and reproduce that outcome from saved configuration and seed.

Submission position: Archis supplies the required virtual camera tracking workflow, a passing local test suite and current verified benchmark evidence. Before final release, repeat the speed matrix and package smoke test on Windows, and score an independent noisy 30 FPS MP4 with truth. The jitter pass is specifically the declared inertial-assisted configuration.

References

- [1] Smart India Hackathon Problem Statement 26169, Department of Space / ISRO, supplied problem statement.
- [2] Archis Optical Tracker 2.0 local source based on commit 0143499; 25 September 2026 verified benchmark evidence, engine fingerprint ad8cd198e6ee.
- [3] L. C. Andrews and R. L. Phillips, *Laser Beam Propagation through Random Media*, 2nd ed., SPIE Press, 2005.
- [4] M. S. Grewal and A. P. Andrews, *Kalman Filtering: Theory and Practice Using MATLAB*, 4th ed., Wiley, 2015.
- [5] J. W. Goodman, *Introduction to Fourier Optics*, 4th ed., W. H. Freeman, 2017.
- [6] OpenCV project documentation, DNN and image-processing modules used by the implementation.

End of report